

Helix Compute

Sparse Dependency-Aware Execution for Incremental Distributed Systems

Abstract

Modern distributed systems optimize storage, transport, and state management, yet still frequently recompute large dependency graphs globally even when only small portions of system state have changed. Helix Compute introduces a sparse dependency-aware execution model that:

- detects localized mutation
- performs reverse dependency indexing
- replays only affected execution neighborhoods
- preserves deterministic reconstruction

Across topology, mutation, and replay-stress benchmark sweeps, Helix maintained stable propagation locality while substantially reducing unnecessary recomputation. The primary result is not raw speed alone, but bounded locality preservation under incremental mutation.

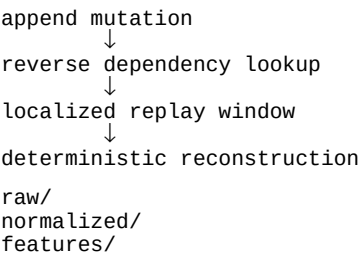
Industry Context

Helix operates as an execution-layer optimization that complements existing storage, transport, and versioning systems. Rather than replacing existing infrastructure, Helix reduces unnecessary downstream recomputation while preserving deterministic replay behavior.

System	Primary Optimization
Parquet	Storage locality and compression
Delta Lake	State/version tracking
Kafka / Flink	Event transport and streaming
Datadobi	Migration and reconciliation
Helix Compute	Sparse dependency-aware recomputation

Runtime Architecture

Traditional pipelines frequently execute global recomputation after incremental change. Helix instead performs localized replay propagation:



- raw/ preserves source ingestion state
- normalized/ preserves canonical aligned historians
- features/ preserves replay-derived state

Reverse Dependency Indexing

Traditional propagation systems repeatedly scan full dependency graphs during expansion traversal. Helix replaces repeated global scans with reverse dependency indexing.

Metric	Observed Result
Dataset Size	100,000 records
Mutation Set	2,000 records
Validation Match	True
Traditional Expansion Time	22.2656s
Indexed Expansion Time	0.0019s
Observed Speedup	11,940.78x

Sparse Replay Validation

Sparse replay was validated against a real append-heavy runtime environment.

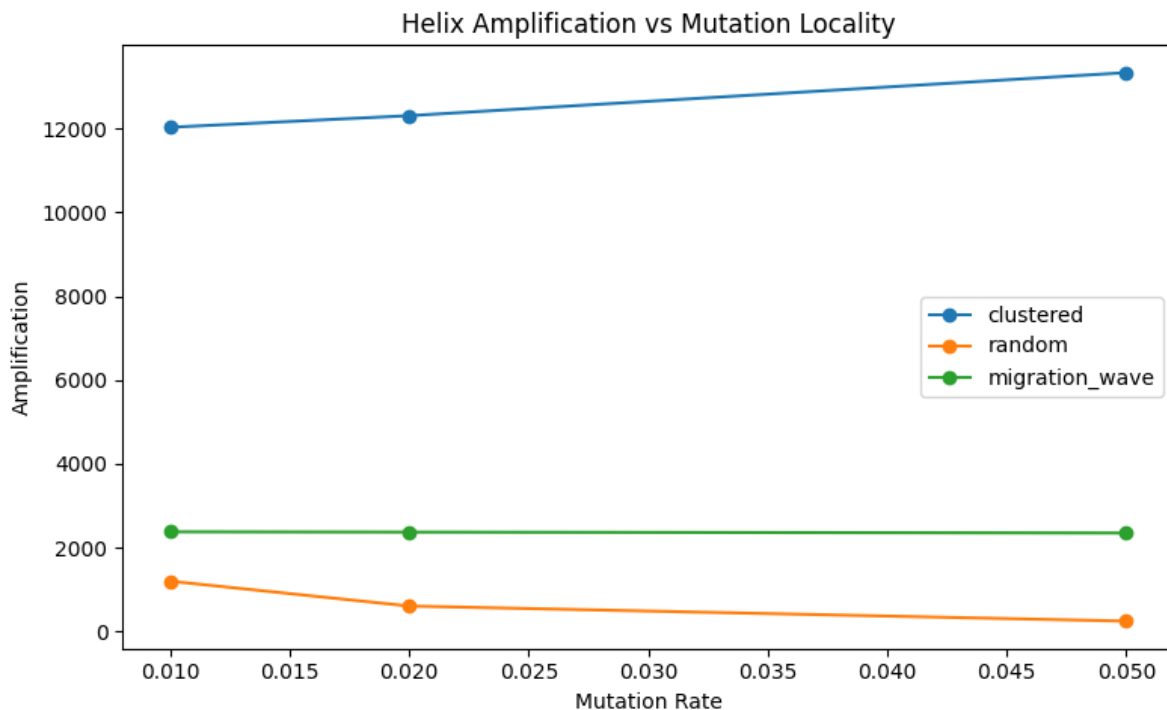
Total Rows:
1,027,581

Touched Nodes:
30

Validation:
MATCH: True

Observed Replay Speedup:
16.04x

Execution Compression vs Mutation Locality



Key Findings

1. Dependency Expansion Remained Bounded

Across most workloads:

dependency radius ≈ 1.2 – 1.4

2. Reverse Indexing Removed Traversal Bottlenecks

Observed indexed propagation speedups exceeded 11,000x while preserving deterministic validation.

3. Clustered Mutation Produced Maximum Efficiency

Execution Compression Factor: 13,333x

Reduction: 99.9925%

4. Replay Storms Did Not Produce Catastrophic Propagation

Replay amplification remained bounded even under replay-stress conditions.

Operational Implications

Helix Compute is designed for systems where:

- data changes incrementally
- recomputation dominates runtime cost
- deterministic reconstruction matters
- replay locality remains bounded

These conditions commonly appear in telemetry pipelines, append-heavy historian systems, CDC workflows,

streaming ETL systems, and time-series infrastructure. Rather than replacing storage or streaming systems, Helix reduces unnecessary downstream recomputation by preserving dependency locality during execution.

Conclusion

The benchmark results presented in this paper demonstrate that:

- reverse dependency indexing

- sparse replay propagation
- localized execution neighborhoods
- deterministic reconstruction

can substantially reduce unnecessary recomputation in incremental distributed systems. Across topology, mutation, and replay-stress sweeps, Helix maintained stable propagation behavior while preserving validation equivalence.

The most important result is not peak amplification, but sustained locality preservation under incremental mutation. Helix complements existing systems such as Parquet, Delta Lake, Kafka, and Flink by optimizing execution locality rather than storage, transport, or state versioning.